Software Engineering I (02161)

Introduction to Software Engineering

Assoc. Prof. Hubert Baumeister

DTU Compute
Technical University of Denmark

The course

- 5 ECTS course 02161: Software Engineering 1
  - Approx. 9 hours per week
- Learning objectives
  - Tools and methods to help you create software
  - We use agile software development
  - Do a *smaller* project from an *informal* and *open description* of the problem

Who are we?

- Approx 136 students with different backgrounds
  - Bachelor Software Technology: 60%
  - Bachelor Cybertechnology: 15%
  - Bachelor Computer Engineering: 14%
  - Other: 11%
- Teacher
  - Hubert Baumeister, Assoc. Prof. at DTU Compute (huba@dtu.dk; office 321.025)
- 3 Teaching assistants
  - Sebastian Thyregod Svendsen (s235244@student.dtu.dk)
  - Anton Moth Liston (s235256@student.dtu.dk)
  - Emil Breinholt (s235273@student.dtu.dk)

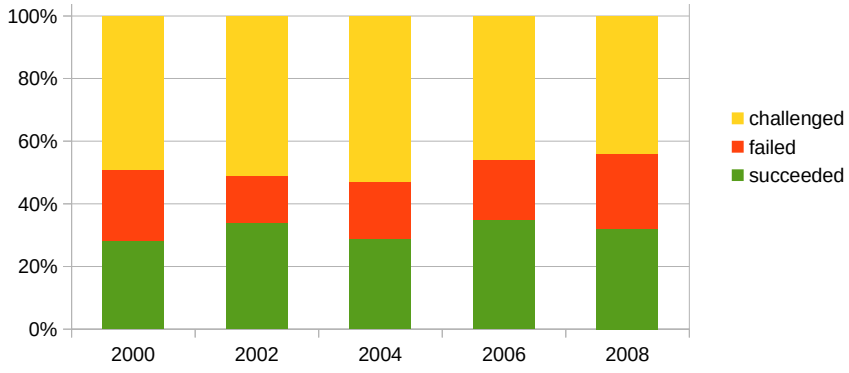
Software project challenges

Challenges: On time, within budget, without bugs, and does what the customer needs it to do

Failed or challenged

- Amanda
- Rejsekortet
- Obamacare Website
- German road toll system
- ...

Success rate for software projects 2000—2008



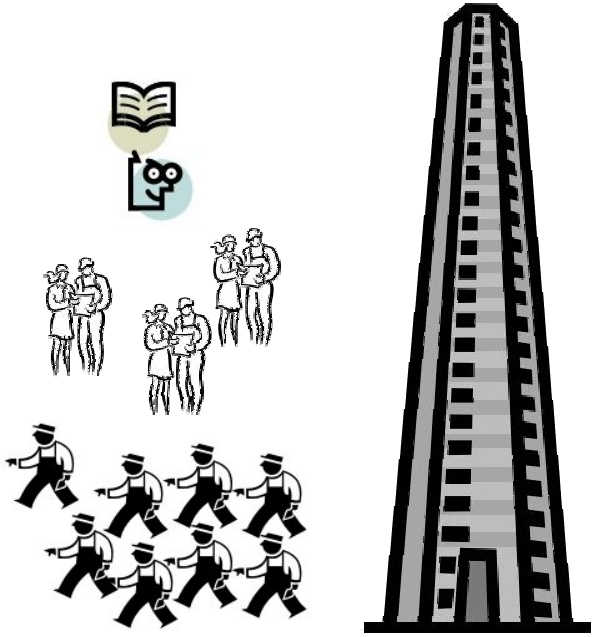
CHAOS Summary 2009 Report

- Succeeded: 32%
- Failed: 20%
- Challenged: 48% (over time, over budget, ...)

Scaling software development

Small hut

- one person
- no special knowledge



Skyscraper

- not possible with one person
- special knowledge: static, electricity, water, waste, elevator, …

Small software — large software: bug fixing

Small program

- find the defect
- fix the defect
- (adjust documentation)

Large software

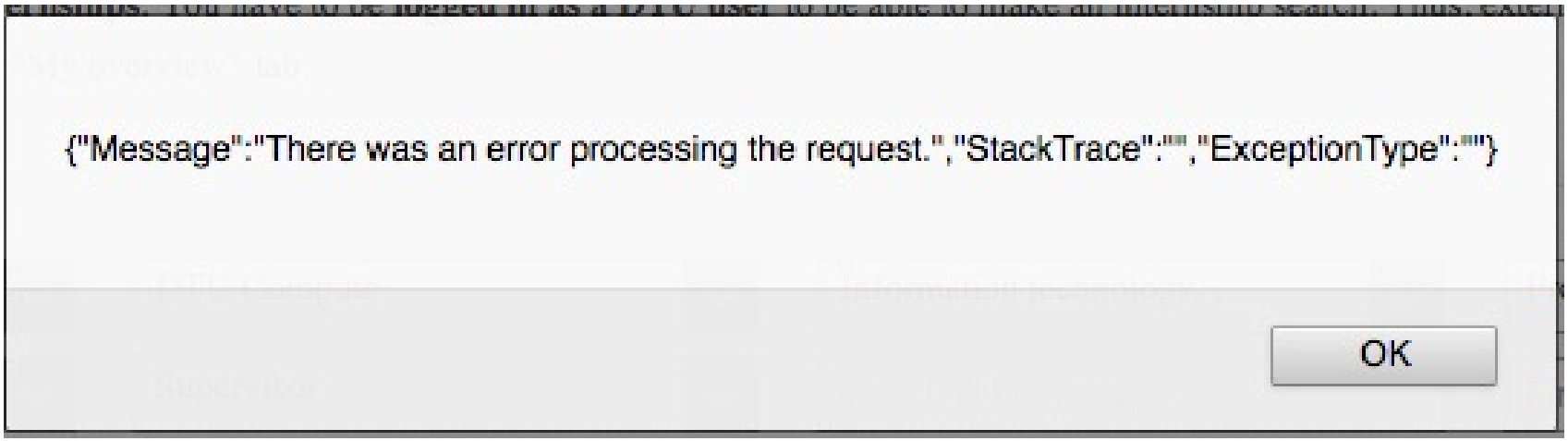
- report defect
- collect defect reports
- analyse problem
- identify bug
- define a bug fixing strategy
- fix the bug
- testing: bug fixed; no new bugs
- accept the fixed version
- integrate parallel changes
- update release documentation
- release the new system

Software development is more than programming

- Lines of code per day: 10 — 50
- Debugging (finding and fixing bugs)
- Refactoring (deleting code)
- Validation (e.g. tests)
- Documentation (User–, System–)
- Configuration files
- ...

Software attributes

- Maintainability
  - Reduce complexity
  - Readable code
- Dependability and security
  - Reliability, privacy, and safety
  - Example: Apple root access on macOS
- Acceptability / user friendliness



- Efficiency

Software *Engineering*

Young disciplin: 1968 Nato conference

Software Engineering Definition (Sommerville 2010)

> Software engineering is an *engineering discipline*
> that is concerned with *all aspects* of **software production**
> from the early stages of system specification
> through to maintaining the system after it has gone into use

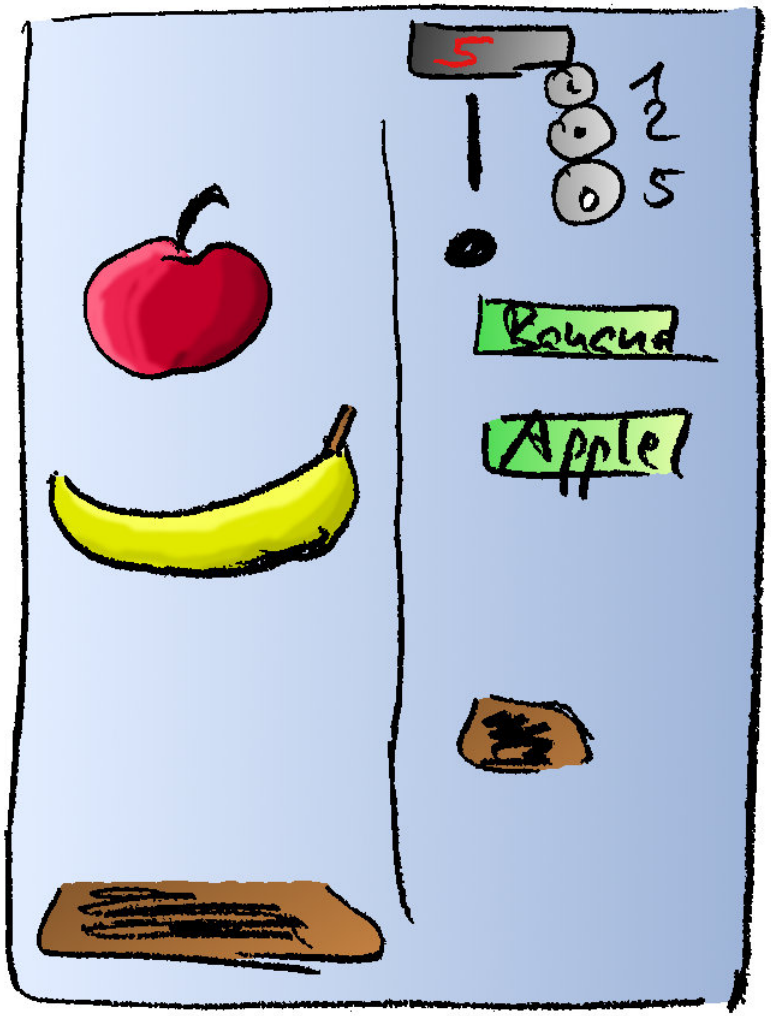
An engineer applies **theories**, **methods**, and **tools**

Theories, Tools, and Methods

Basic Activities in Software Development

- *Requirements Analysis*
  - Use cases, user stories, domain model
- *Software Design*
  - Class diagrams, sequence diagrams, CRC cards, design patterns
- *Implementation*
  - Good design principles, layered architecture, S.O.L.I.D principles, design by contract
- *Test*
  - automated vs manual tests, Test- and behvaiour driven development (TDD/BDD), black-box and white-box tests, code coverage

Example Vending Machine Controller



Controller software for a vending machine

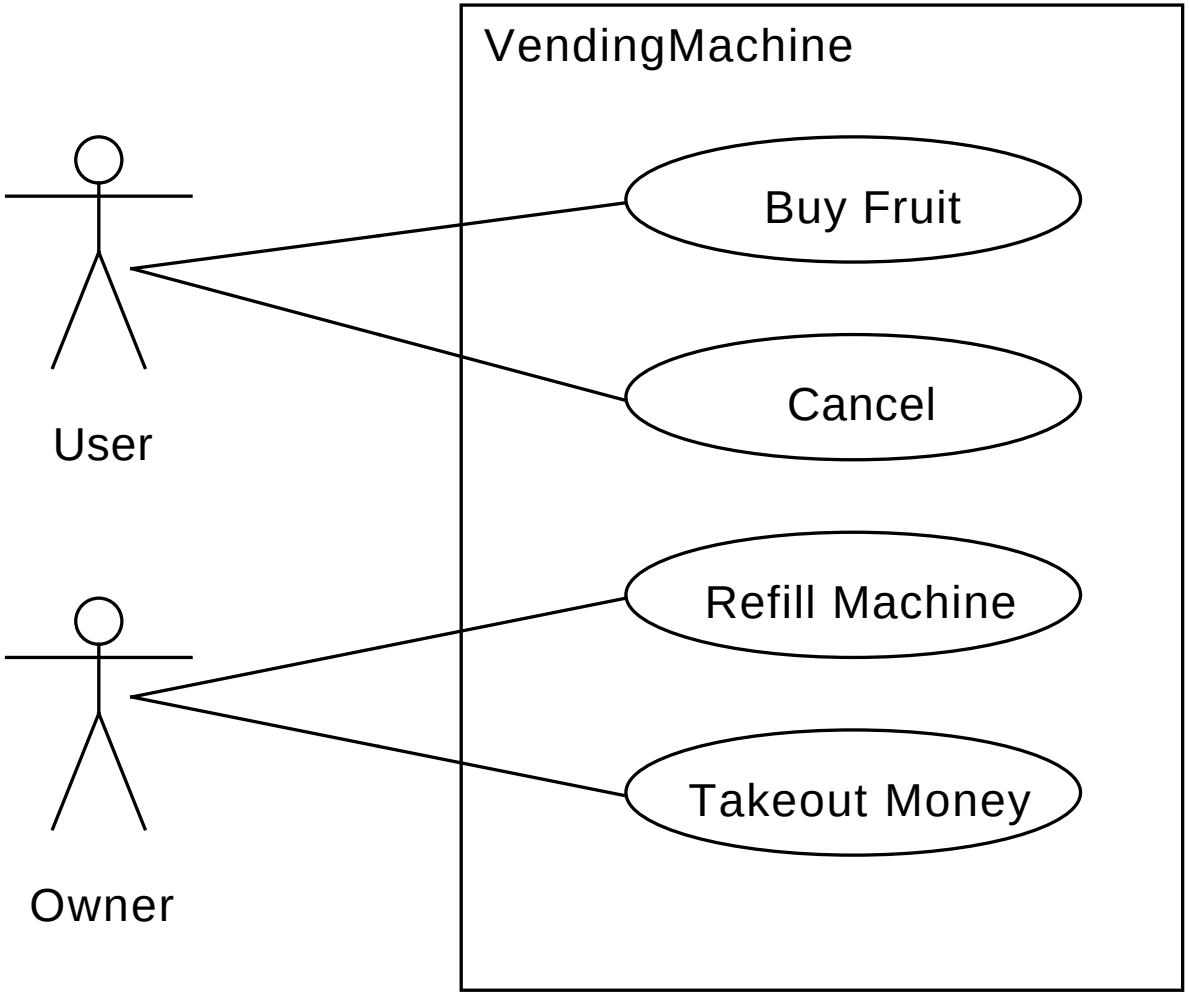
Requirements: Glossary

Example

- Vending machine: The vending machine allows users to *buy fruit*.
- User: The user of the *vending machine* buys fruit by inserting coins into the machine.
- Owner: The owner owns the *vending machine*. He is required to refill the machine and can remove the money from the machine.
- Display: The display shows how much money the *user* has inserted.

  ...

Requirements: Use case diagram



Requirements: Detailed Use Case: Buy Fruit

```
Feature: Buy fruit
   Description: A user buys a fruit from the vending machine
   Actors: User

Scenario: Buy a fruit with enough money
   Given the vending machine has fruits
   When the user enters enough money for a fruit
   And the user selects a fruit
   Then the fruit will be dispensed
   And the machine returns the rest money
   And the machine remembers its earnings

... (More scenarios)
```

Validation: Specify success criteria: *Acceptance tests*

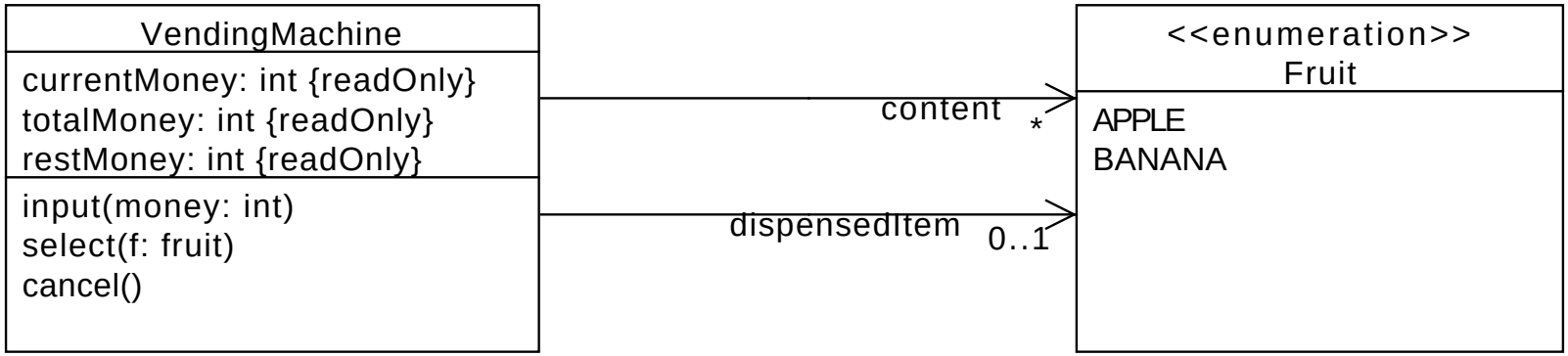
Use detailed use cases directly (Cucumber)

```
Scenario: Buy a fruit with enough money
   Given the vending machine has fruits
   When the user enters enough money for a fruit
   And the user selects a fruit
   Then the fruit will be dispensed
```

VendingMachineSteps.java

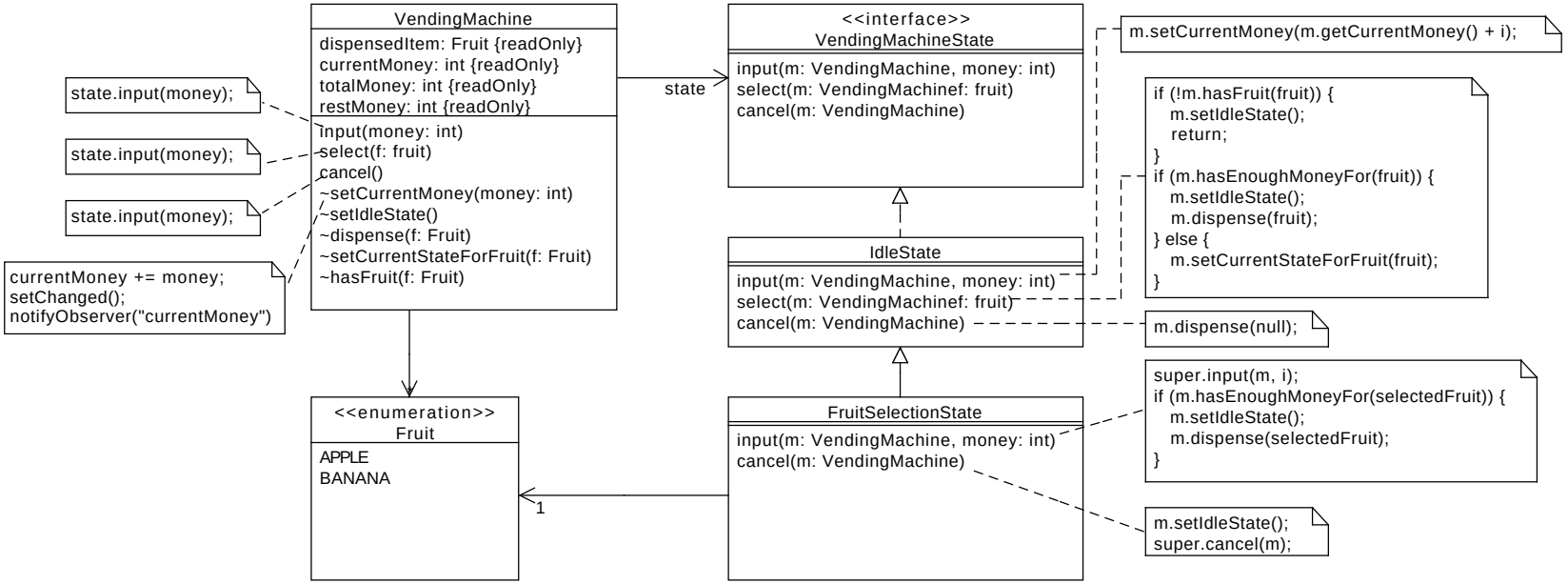
```
@Given("the vending machine has fruits")
public void theVendingMachineHasFruits() throws Exception {
   vendingMachine = new VendingMachine(2,2);
}
@When("the user enters enough money for a fruit")
public void theUserEntersEnoughMoneyForAFruit() throws Exception {
   vendingMachine.input(3);
}
@When("the user selects a fruit")
public void theUserSelectsTheFruit() throws Exception {
   vendingMachine.selectFruit(Fruit.APPLE);
}
@Then("the fruit will be dispensed")
public void theFruitWillBeDispensed() throws Exception {
   assertEquals(Fruit.APPLE, vendingMachine.getDispensedItem());
}
```

Design: High-level Class diagram



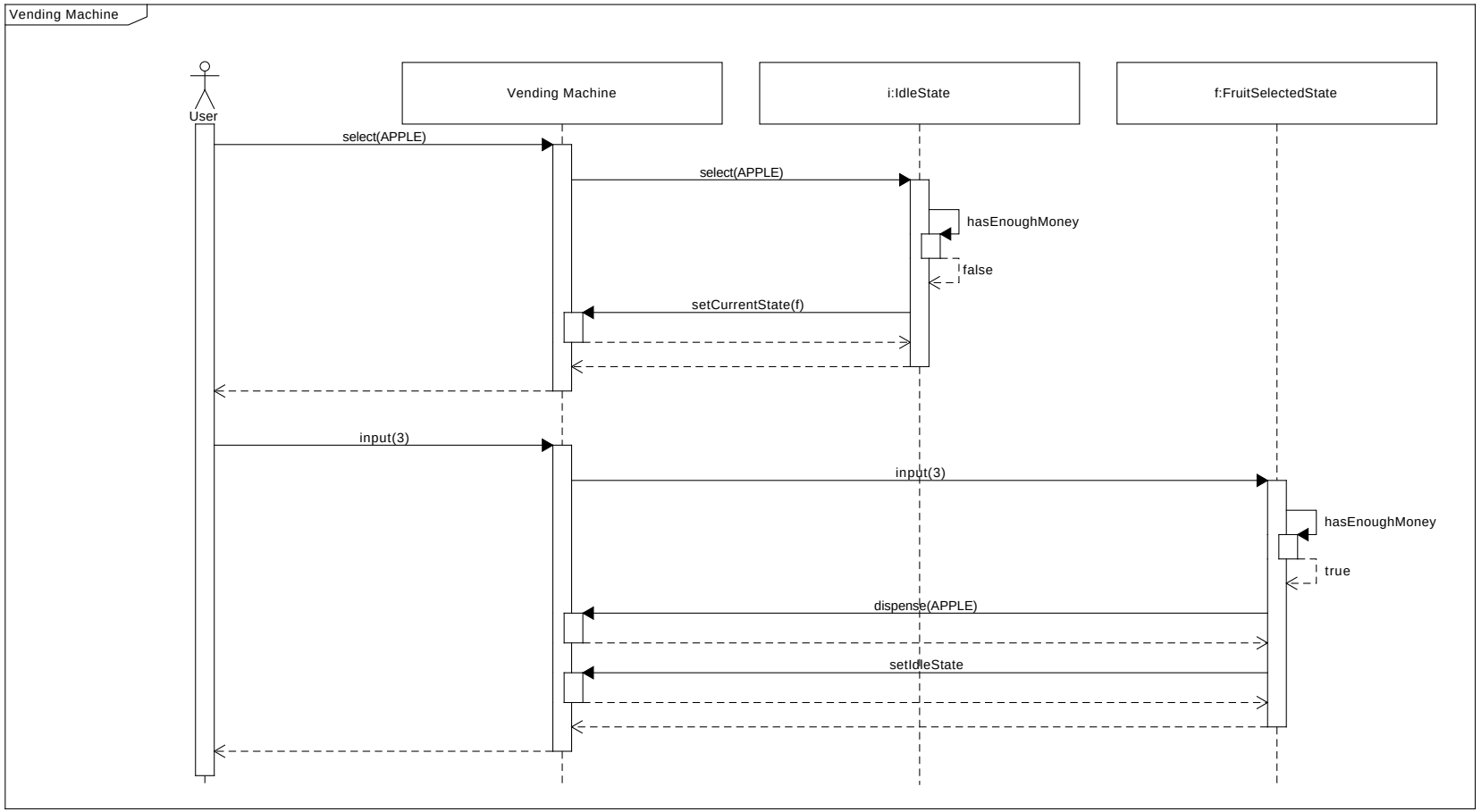
Design: Design of the system as class diagram

Use State pattern (a *design pattern*)



Design: Visualization of the Execution

- *Interaction Diagrams*, aka. *Sequence Diagrams*
  - used for designing the system
  - used for documenting the system
- Vending Machine



Course content

- (w1) Introduction to Software Engineering and TDD (BDD)
- (w2–w3) Agile Software Development and Software Testing
  - Test Driven Development
  - Systematic Tests, Code Coverage
  - Programming exercises
- (w4–w6) Modelling
  - Requirements Engineering (Use Cases, User Stories, Domain Model)
  - System Modelling (Class Diagrams, Sequence Diagrams)
  - Programming exercises cont. and modelling exercises
- (w7–w10) Design and Implementation + Exam Project Start
  - Architecture (hexagonal, layered, . . .)
  - Design (SOLID, Design Patterns, Design by Contract, . . .)
- (w11–w12) Exam project
- (w13) Project Presentations; Exam project submission

Software and Tools

- Programming language: Java (JDK 21 or higher): Recommended JDK 25 LTS (Long Term Support version)
- IDE:
  - Eclipse (Simple Java IDE is sufficient) (Is used in videos and also quite often by me)
  - IntelliJ (again CE is sufficient, but Ulltimate is available for DTU students)
  - Visual Studio Code (VS Code)
- Test framework: Cucumber, JUnit, Mockito

...

Use of AI

- AI: E.g. CoPilot, ChatGPT, DeepSeek, . . .
- Use is allowed but, its use needs to be documented in the exam project (in the report and in the source code)
- Use AI like an expert, e.g., like a TA or a teacher
  - Can you explain the following concept?
  - Can you explain that error message?
  - Can you show me how to iterate over a list in Java?
  - . . .
  - One possible use of copilot
    - TDD: Ask AI to write tests and then the production code
    - Ask the AI to do the refactorings you want.
    - **red-green-refactor**
  - Give me some exercises to teach me this concept

Course activities: ”Flipped Class Room”

- At home: study videos and teaching material:
  - See the Videos
    - they are available starting Tuesday evening the week before
  - Exercises
  - Ask questions
    - via the discussion feature on DTU Learn (e.g., Muddy Points)
- During the Lecture: Q&A, exercises with solutions, work with TA present
  - Muddy Points (answer to muddy points raised in the Muddy Points Discussion Topic)
  - Small group exercise and discussing the solution together
  - Exercises (one larger programming exercise; small exercises for other topics) (not graded)
    - done during the lecture time and outside
- TA’s and I will be present
  - Help with the exercises
- Ask questions on the DTU Learn disussion forum at other times

Examination

- Exam project in groups of 4-5
  - Model, Software, Reports, Peer Review, Demonstration
  - → Focus on that you have learned the techniques and methods
  - *no* written/oral examination
- Week 5–6: Forming of project groups
- Week 6: Start of exam project
- Week 8: First delivery (modelling): requirements and first design
- Week 9: Peer review of another groups model
- Week 10 - 13: Implementation and report writing
- Monday week 13: Demonstration of the projects (each project 10 min) and submission of final implementation

→ Grade is based on an evaluation of submissions (reports, peer review, source code) as a whole (helheds vurdering)

Course material

- DTU Learn contains
  - practical information: (e.g. lecture plan)
  - Course material (e.g. videos, slides, exercises, notes)
  - Check regularly
- DTU Learn is being used to send announcements
  - *make sure you are notified*
- DTU Learn discussion for answers to questions
- Books:
  - Suplementary resources
    - I recommend Uncle Bob Video lecture on Aspects of Software Development (links on DTU Learn)
    - Book ”Soft Engineering Basic Principles and Best Practices” by Ravi Sethi, Cambridge University Press, 2023
    - Additional literature on DTU Learn